

Manual de Utilização: Gerador de Massa de Dados QA & Testes de API

Este manual detalha a utilização dos módulos do projeto, que combina um gerador de massa de dados com testes de API para o JSONPlaceholder, utilizando Jest, Axios e Postman.

1. Módulo de Geração de Massa de Dados

Este módulo é responsável por criar dados realistas e dinâmicos para seus testes, utilizando a biblioteca `faker-js`.

1.1. Estrutura dos Geradores

Os geradores estão localizados na pasta `generators/`:

- `userGenerator.js` : Gera dados de usuários (nome, e-mail, endereço, etc.).
- `productGenerator.js` : (Se implementado) Gera dados de produtos.

1.2. Como Utilizar os Geradores

Você pode usar os geradores diretamente em seus scripts ou testes. Veja um exemplo de como instanciar e gerar dados de usuário:

```
const UserGenerator = require("../generators/userGenerator");

// Instancia o gerador com locale 'pt_BR' e uma semente para
reprodutibilidade
const userGen = new UserGenerator("pt_BR", 12345);

// Gera um único usuário
const singleUser = userGen.generateOne();
console.log("Usuário único:", singleUser);

// Gera uma lista de 5 usuários
const usersList = userGen.generate(5);
console.log("Lista de usuários:", usersList);
```

1.3. Execução via Script Principal

O script `scripts/runGeneration.js` é o ponto de entrada para gerar massa de dados de forma automatizada, conforme configurado em `config/settings.json`.

Para executar:

```
npm start
```

Os dados gerados serão exportados para a pasta `data/output/` nos formatos especificados.

2. Módulo de Testes de API (Jest & Axios)

Este módulo contém os testes automatizados para a API do JSONPlaceholder, escritos em JavaScript utilizando Jest para o framework de testes e Axios para as requisições HTTP.

2.1. Estrutura dos Testes

Os testes de API estão organizados na pasta `tests/api/`:

- `jsonplaceholder.test.js` : Contém testes unitários para os endpoints básicos do JSONPlaceholder (GET, POST, PUT, DELETE para `/posts` e `/users`).
- `integration.test.js` : Demonstra a integração do gerador de massa de dados com os testes de API, onde dados gerados dinamicamente são usados para criar e manipular recursos na API.

2.2. Como Executar os Testes

Certifique-se de ter as dependências instaladas (`npm install`). Para rodar todos os testes do projeto (incluindo os testes de geradores e de API):

```
npm test
```

Os resultados serão exibidos no terminal, indicando quais testes passaram ou falharam.

2.3. Entendendo os Testes

Cada arquivo de teste (`.test.js`) utiliza a sintaxe do Jest (`describe` , `test` , `expect`). As requisições HTTP são feitas com `axios` . Os testes verificam:

- **Status HTTP:** Se a resposta da API retorna o código esperado (ex: 200 OK, 201 Created).
- **Estrutura da Resposta:** Se a resposta contém os campos esperados.
- **Dados da Resposta:** Se os dados retornados correspondem aos dados enviados ou esperados.

3. Módulo Postman Collection e Environment

Para facilitar a execução manual, depuração e colaboração, o projeto inclui uma Postman Collection e um Environment pré-configurados.

3.1. Localização dos Arquivos

Os arquivos estão na pasta `postman/` :

- `JSONPlaceholder_Collection.json` : A coleção de requisições para a API.
- `JSONPlaceholder_Environment.json` : O ambiente com variáveis como `base_url`.

3.2. Como Importar para o Postman

1. Abra o aplicativo Postman.
2. Clique em `File > Import` (ou no botão `Import` na interface).
3. Selecione ambos os arquivos `.json` da pasta `postman/` do seu projeto.
4. Após a importação, a coleção `JSONPlaceholder API Tests` e o ambiente `JSONPlaceholder - Dev` estarão disponíveis.

3.3. Utilizando a Collection

1. **Selecione o Ambiente:** No canto superior direito do Postman, selecione `JSONPlaceholder - Dev` no dropdown de ambientes.
2. **Explore as Requisições:** A coleção contém pastas para `Posts` e `Users`, com requisições GET, POST, PUT, DELETE configuradas.
3. **Execute os Testes:** Cada requisição possui testes pré-configurados na aba `Tests`. Ao enviar a requisição, os testes serão executados automaticamente e os resultados exibidos.

3.4. Variáveis de Ambiente

O ambiente `JSONPlaceholder - Dev` define a variável `base_url` como `https://jsonplaceholder.typicode.com`. Você pode modificar esta variável ou adicionar outras conforme necessário para diferentes ambientes (ex: staging, produção).

Desenvolvido por Manus AI